

Array Operations in NumPy

AI Launchpad Program – Data Science with Python

Introduction

Once an array is created, the next step is to perform operations on it. One of the biggest advantages of NumPy is that it allows mathematical operations to be performed directly on arrays without using loops. This concept is known as **vectorization**, making NumPy code faster, cleaner, and more efficient than traditional Python code.

In this chapter, we will explore various operations that can be performed on NumPy arrays.

Learning Objectives

By the end of this chapter, you will be able to:

- Perform arithmetic operations on arrays.
 - Apply scalar operations.
 - Compare arrays.
 - Use logical operations.
 - Perform aggregate operations like sum and average.
 - Find minimum and maximum values.
 - Understand vectorized operations.
 - Use mathematical functions on arrays.
-

Creating Sample Arrays

```
import numpy as np
```

```
a = np.array([10, 20, 30, 40])
```

```
b = np.array([1, 2, 3, 4])
```

```
print(a)
```

```
print(b)
```

Output

```
[10 20 30 40]
```

```
[1 2 3 4]
```

1. Addition of Arrays

Corresponding elements are added together.

```
import numpy as np
```

```
a = np.array([10,20,30])
```

```
b = np.array([1,2,3])
```

```
result = a + b
```

```
print(result)
```

Output

```
[11 22 33]
```

Equivalent function

```
result = np.add(a, b)
```

```
print(result)
```

Output

```
[11 22 33]
```

2. Subtraction of Arrays

```
import numpy as np
```

```
a = np.array([20,30,40])
```

```
b = np.array([5,10,15])
```

```
print(a - b)
```

Output

```
[15 20 25]
```

Using function

```
np.subtract(a, b)
```

3. Multiplication

Element-wise multiplication

```
import numpy as np
```

```
a = np.array([2,4,6])
```

```
b = np.array([3,5,7])
```

```
print(a * b)
```

Output

```
[ 6 20 42]
```

Using function

```
np.multiply(a, b)
```

4. Division

```
import numpy as np
```

```
a = np.array([20,40,60])
```

```
b = np.array([2,4,5])
```

```
print(a / b)
```

Output

```
[10. 10. 12.]
```

Using function

```
np.divide(a, b)
```

5. Exponent (Power)

```
import numpy as np
```

```
a = np.array([2,3,4])
```

```
print(a ** 2)
```

Output

```
[ 4  9 16]
```

Using function

```
np.power(a, 2)
```

6. Modulus Operation

Returns the remainder.

```
import numpy as np
```

```
a = np.array([10,20,30])
```

```
print(a % 4)
```

Output

```
[2 0 2]
```

Using function

```
np.mod(a, 4)
```

7. Scalar Operations

A scalar is a single number. NumPy automatically applies the operation to every element.

Addition

```
import numpy as np
```

```
arr = np.array([10,20,30])
```

```
print(arr + 5)
```

Output

```
[15 25 35]
```

Multiplication

```
print(arr * 2)
```

Output

```
[20 40 60]
```

Division

```
print(arr / 10)
```

Output

```
[1. 2. 3.]
```

8. Comparison Operations

Greater Than

```
import numpy as np
```

```
arr = np.array([5,10,15,20])
```

```
print(arr > 10)
```

Output

```
[False False True True]
```

Less Than

```
print(arr < 15)
```

Output

```
[ True True False False]
```

Equal To

```
print(arr == 10)
```

Output

```
[False True False False]
```

Not Equal

```
print(arr != 20)
```

Output

```
[ True True True False]
```

9. Logical Operations

Logical AND

```
import numpy as np
```

```
a = np.array([True, False, True])
```

```
b = np.array([True, True, False])
```

```
print(np.logical_and(a, b))
```

Output

```
[ True False False]
```

Logical OR

```
print(np.logical_or(a, b))
```

Output

```
[ True True True]
```

Logical NOT

```
print(np.logical_not(a))
```

Output

```
[False True False]
```

10. Aggregate Operations

Aggregate functions summarize the entire array.

Sum

```
import numpy as np
```

```
arr = np.array([10,20,30,40])
```

```
print(arr.sum())
```

Output

```
100
```

or

```
np.sum(arr)
```

Mean (Average)

```
print(arr.mean())
```

Output

25.0

or

```
np.mean(arr)
```

Maximum

```
print(arr.max())
```

Output

40

Minimum

```
print(arr.min())
```

Output

10

Standard Deviation

```
print(arr.std())
```

Output

11.18033989

Variance

```
print(arr.var())
```

Output

125.0

11. Product of Elements

```
import numpy as np
```

```
arr = np.array([1,2,3,4])
```

```
print(arr.prod())
```

Output

```
24
```

12. Cumulative Sum

```
import numpy as np
```

```
arr = np.array([1,2,3,4])
```

```
print(arr.cumsum())
```

Output

```
[ 1  3  6 10]
```

13. Cumulative Product

```
import numpy as np
```

```
arr = np.array([1,2,3,4])
```

```
print(arr.cumprod())
```

Output

```
[ 1  2  6 24]
```

14. Absolute Values

```
import numpy as np
```

```
arr = np.array([-10,-20,15])
```

```
print(np.abs(arr))
```

Output

```
[10 20 15]
```

15. Square Root

```
import numpy as np
```

```
arr = np.array([4,9,16,25])
```

```
print(np.sqrt(arr))
```

Output

```
[2.  3.  4.  5.]
```

16. Trigonometric Functions

Sine

```
import numpy as np

angles = np.array([0, np.pi/2, np.pi])

print(np.sin(angles))
```

Output

```
[0.  1.  0.]
```

Cosine

```
print(np.cos(angles))
```

Output

```
[ 1.  0. -1.]
```

17. Rounding Functions

```
import numpy as np

arr = np.array([2.34, 4.78, 6.12])

print(np.round(arr))
```

Output

```
[2.  5.  6.]
```

Floor

```
print(np.floor(arr))
```

Output

```
[2. 4. 6.]
```

Ceiling

```
print(np.ceil(arr))
```

Output

```
[3. 5. 7.]
```

18. Element-wise Operations on 2D Arrays

```
import numpy as np
```

```
a = np.array([  
    [1,2],  
    [3,4]  
])
```

```
b = np.array([  
    [5,6],  
    [7,8]  
])
```

```
print(a + b)
```

Output

```
[[ 6  8]
```

```
[10 12]]
```

Multiplication

```
print(a * b)
```

Output

```
[[ 5 12]
```

```
[21 32]]
```

19. Matrix Multiplication

Element-wise multiplication is different from matrix multiplication.

```
import numpy as np
```

```
a = np.array([
```

```
    [1,2],
```

```
    [3,4]
```

```
])
```

```
b = np.array([
```

```
    [5,6],
```

```
    [7,8]
```

```
])
```

```
print(np.matmul(a,b))
```

Output

```
[[19 22]
```

```
[43 50]]
```

Using operator

```
print(a @ b)
```

Output

```
[[19 22]
```

```
[43 50]]
```

20. Vectorized Operations

Without NumPy

```
numbers = [1,2,3,4,5]
```

```
result = []
```

```
for i in numbers:
```

```
    result.append(i * 2)
```

```
print(result)
```

Output

```
[2,4,6,8,10]
```

Using NumPy

```
import numpy as np
```

```
arr = np.array([1,2,3,4,5])
```

```
print(arr * 2)
```

Output

```
[ 2  4  6  8 10]
```

Notice how no loop is required.

Why Vectorization is Faster

Vectorized operations:

- Reduce Python loops.
 - Use optimized C implementations internally.
 - Consume less memory.
 - Improve CPU cache utilization.
 - Execute significantly faster on large datasets.
-

Real-Life Example

Suppose the prices of products increase by **10%**.

```
import numpy as np
```

```
prices = np.array([100,250,400,500])
```

```
new_prices = prices * 1.10
```

```
print(new_prices)
```

Output

```
[110. 275. 440. 550.]
```


No loop is needed to update every price.

Common Mistakes

Arrays Must Have Compatible Shapes

```
a = np.array([1,2,3])
```

```
b = np.array([4,5])
```

```
print(a + b)
```

Output

ValueError:

operands could not be broadcast together

Both arrays must have compatible dimensions (or follow broadcasting rules, covered in a later chapter).

Best Practices

- Prefer vectorized operations over loops.
 - Use built-in NumPy mathematical functions.
 - Ensure arrays have compatible shapes before performing operations.
 - Use `@` or `np.matmul()` for matrix multiplication instead of `*`.
 - Leverage aggregate functions for quick statistical summaries.
-

Summary

In this chapter, you learned:

- Arithmetic operations on arrays
 - Scalar operations
 - Comparison and logical operations
 - Aggregate functions (`sum`, `mean`, `min`, `max`, `std`, `var`)
 - Product and cumulative operations
 - Mathematical functions (`sqrt`, `abs`, `sin`, `cos`)
 - Rounding functions
 - Element-wise vs. matrix multiplication
 - Vectorized operations and their advantages
-

Key Takeaways

- NumPy performs operations **element-wise** by default.
- **Vectorization** eliminates the need for explicit loops, resulting in cleaner and faster code.
- Aggregate functions make it easy to compute statistics from large datasets.
- Use `*` for element-wise multiplication and `@` (or `np.matmul()`) for matrix multiplication.
- Mastering array operations is essential before moving on to advanced topics like **array indexing, slicing, broadcasting, reshaping, and linear algebra**.
-